

審查系統設計文件

技術架構

- **後端**: Node.js + Express + TypeScript
- **NLP**: Gemini API (文字切割、標註、Sub-criteria 命名)
- **Embedding**: Gemini Embedding API
- **Vector DB**: ChromaDB 或 Qdrant (本地 Docker)
- **Criteria**: 由校務會議預先定義, 共 4 個, 系統不自動探索

預定義 Criteria

編號	名稱
C1	學術根基與跨域修課表現 (Academic Foundation & Interdisciplinary Learning)
C2	專題實作與技術應用能力 (Project Implementation & Technical Skills)
C3	問題解決與批判性思考 (Problem Solving & Critical Thinking)
C4	專業傳遞與自我成長規劃 (Knowledge Transfer & Career Roadmap)

輸入文件類型

[看交大資工所推甄簡章](#)

- **自傳**
- **讀書計畫**
- **其他加分文件** (得獎紀錄、課堂作品、專題報告等)

注意: 不同候選人文件結構差異大 (段落式 vs 條列式 vs OCR 掃描件) 。
Pre-processing 階段需清理 OCR 噪音後再進行 Idea Unit 切割。

Pipeline 流程

步驟 0 — Pre-processing

- 清理 OCR 噪音 (亂碼、錯位數字、掃描殘影)
- 保留文件結構標記 (自傳 / 讀書計畫 / 加分文件) 供後續 LLM 參考

步驟 1 — Idea Unit 分割

[看1.json](#)

- 輸入: 申請者文件 (已清理)
- 用 LLM 根據語意將文件切割成 Idea Units
- 每個 Idea Unit 必須語意完整、不可再分
- 對「活動紀錄型條列文字」需特別處理: 每條活動切為獨立 Idea Unit, 避免語意過薄

步驟 2 — Seed Batch 標註 (前 30 份文件)

對每個 Idea Unit, LLM 執行三件事:

1. 映射到 1-2 個 Criteria
2. 對每個映射到的 Criterion，各自在該 Criterion 的範疇內自由命名一個 raw Sub-criteria label
3. 提取 Hashtag 關鍵字（全體共用，不分 Criterion）

設計決策：一個 Idea Unit 若映射到 2 個 Criteria，則建立 2 個 Sub-criteria label（每個 Criterion 各一個），因為同一內容透過不同 Criterion 視角會產生不同的語意框架。

例：「在產學合作中帶領同學用 ML 解決真實問題」

- C2 視角 → 產學實作與ML技術落地
- C3 視角 → 真實場景問題建構能力

輸出：每個 Idea Unit 帶有 `criteria[]`、`sub_criteria_map: { [criterion]: raw_label }`、`hashtags[]`

步驟 3 — Sub-criteria 收斂 (Hybrid Clustering)

針對每個 Criterion 分別處理：

- **Step 3a：**收集該 Criterion 下所有 30 份文件的 raw sub-criteria labels
- **Step 3b：**對這批 labels 做 embedding，用 BIC-K-Means 找出最佳 K（目標 4-6 群）
- **Step 3c：**將每群的 labels 清單交給 LLM，歸納命名出該群的標準 Sub-criteria
- **Step 3d：**人工 review 確認最終標準字典後鎖定

最終的label不能overlap, mutually exclusive, descriptive enough, actionable, 把偏離點放其他

max: 10

輸出：每個 Criterion 底下 4-6 個標準 Sub-criteria，形成標準字典

Future work:
如果有新的sub-criteria，可以把這些資料統計起來，看要不要新增sub-criteria

步驟 4 — Relabeling（前 30 份文件）

- 將步驟 2 中 30 份文件的所有 raw sub-criteria labels，重新對應至標準字典
- 更新資料庫中每個 Idea Unit 的 `sub_criteria_map` 欄位

步驟 5 — 全量處理（剩餘文件）

- 以步驟 3 鎖定的標準字典，對所有剩餘文件執行與步驟 2 相同的標註流程
- 差異：sub-criteria 直接從標準字典中選取，不再自由命名

步驟 6 — 候選人特徵向量建構

兩種方案皆實作，比較 Silhouette Score 後選用：

方案 A: Embedding + PCA

1. 對每個 Idea Unit，組合文字："Sub-criteria: [標準名稱], Tags: [#tag1, #tag2]"，送入 Gemini Embedding → 1536 維向量
2. 按候選人 × Criterion 分組，對組內所有向量取平均（Mean Pooling）
3. 若某 Criterion 下完全沒有 Idea Unit，以該 Criterion 所有候選人的全局均值補值
4. 4 個 Criterion 均值向量串接 → 6144 維
5. PCA 降維至 60–100 維（保留 80–90% 變異量）後用於分群

方案 B: Sub-criteria 計數矩陣

1. 步驟 3 完成後，標準字典共 16–24 個 sub-criteria（4 Criteria × 4–6 個sub-criteria）

2. 對每個候選人，統計其 Idea Units 中每個標準 sub-criteria 的出現次數 → 16–24 維計數向量
3. L2 正規化（消除因文件量多導致計數偏高的影響）
4. 直接用於分群，無需降維

步驟 7 — 候選人分群

最常用的兩種方式，還要再研究

- 演算法：同時試 **K-Means**（最佳 K 判定：Elbow + Silhouette Score）與 **GMM**（最佳 K 判定：BIC）
- K 搜索範圍：2–6
- 依指標選出最佳演算法與 K 值，為每個候選人標記 `cluster_id`
- 另以 **UMAP 降至 2D**，產生視覺化分布圖（獨立於分群，僅供展示）

步驟 8 — 群體命名與代表人選取

選取 Medoid：在每個群內，找出對群內所有其他成員平均距離最小的候選人，標記為 `is_medoid: true`

群體命名：

1. 統計該 cluster 內所有候選人最高頻的 top-5 sub-criteria 與高頻 hashtags
2. 將此清單交給 LLM，產出 4–8 字的中文群名（如「硬體底層與即時系統」）

步驟 9 — Hashtag 去重

對每個候選人，依以下邏輯產生 `distinctive_hashtags`：

1. 過濾：在所屬 cluster 內出現率 > 70% 的 hashtag 移除
2. 候選：在 cluster 內出現率 < 20% 且該候選人擁有的 hashtag
3. 排序：依 IDF（全體 200 人中越稀少越優先）
4. 輸出：取前 3–4 個

步驟 10 — UI 資料準備

將計算結果寫回 Candidate 模型：

```
{
  candidate_id: "STU_NYCU_001",
  cluster_id: 3,
  cluster_name: "硬體底層與即時系統",
  is_medoid: false,
  distinctive_hashtags: ["#uCOS-II", "#RM_EDF_Scheduling", "#Kernel_Analysis"],
  radar_chart_data: {           // 各 Criterion 下的 Idea Unit 數量（非分數）
    C1: 5,
    C2: 12,
    C3: 8,
    C4: 3
  }
}
```

`radar_chart_data` 使用各 Criterion 的 Idea Unit 數量，代表內容投入廣度，前端標注「各面向內容量」而非分數。

資料模型 (草稿)

Idea Unit

```
{
  id: string,
  candidate_id: string,
  content: string,
  criteria: string[], // 1-2 個 Criterion ID (e.g. ["C1",
"C3"])
  sub_criteria_map: { [criterion: string]: string }, // 每個 Criterion 各自的 Sub-
criteria label
  hashtags: string[], // 全體共用
  embedding?: number[] // 待定 (sub_criteria + hashtags 的
向量)
}
```

Candidate

```
{
  candidate_id: string,
  idea_units: IdeaUnit[],
  // 以下欄位於步驟 6-10 填入
  cluster_id?: number,
  cluster_name?: string,
  is_medoid?: boolean,
  distinctive_hashtags?: string[],
  radar_chart_data?: { C1: number, C2: number, C3: number, C4: number }
}
```

個資保護注意事項

- LLM API 呼叫不得在 prompt 外洩露候選人姓名、學校等識別資訊
- 系統 log 不得記錄原始文件內容
- candidate_id 與真實身份的對應表應獨立儲存，與分析資料庫隔離

限制事項

- **不得產生任何數值分數**供展示
- 所有分群邏輯必須基於向量距離與語意相似度